

Compilers

COMS10015

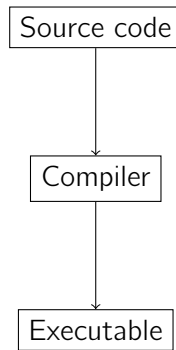
Dr Tom Deakin

University of Bristol

Week 22

Compiler

- ▶ Compilers translate program source code into machine code.
- ▶ Compiler used to translate a high level language.
- ▶ High level languages more complex than assembly. Consist of:
 - ▶ Key words, e.g. `if`, `for`, `while`, ...
 - ▶ Variable names.
 - ▶ Delimiters, e.g. `{}`, `()`, `[]`, ...
 - ▶ Operators, e.g. `+`, `<=`, `==`, ...
 - ▶ Literals/constants, e.g. `123`, `"a string"`, ...
 - ▶ Comments.



Parts of a compiler

Compilers normally broken up into three stages:

1. Front end: Converts source code into an *intermediate representation* (IR).
2. Optimiser: IR transformations, such as optimisation.
3. Back end: Converts IR into machine code.



Compiler parts

- ▶ Breaking up allows a separation of concerns.
- ▶ Front end could translate different languages to a common IR.
- ▶ Back end could translate IR to different machine code.
- ▶ Optimisations often possible on the common IR.
- ▶ This is the approach taken in LLVM.

Front end

Compiler Front end

- ▶ Similar parts to assembler, but must handle additional complexity.
- ▶ Lexer, to break input strings into tokens.
- ▶ Parser (syntax analysis), to build an Abstract Syntax Tree (AST).
- ▶ The AST represents the entire program in memory.
- ▶ The AST is an intermediate representation.
- ▶ Front end may use a number of different intermediate representations at different stages.

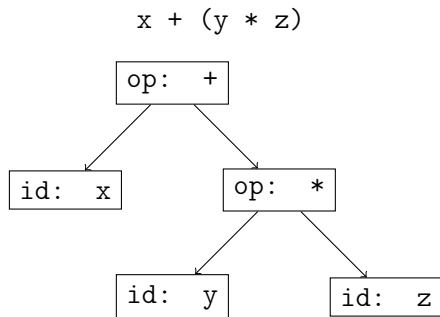
Front end: Lexer

- ▶ The lexer breaks the input program into tokens.
- ▶ Each token represents a symbol or word in the programming language.
- ▶ Might represent this as labelled words.
- ▶ E.g. for `(int i = 0; i < N; ++i)`
(keyword, 'for') (delim, '(') (keyword, 'int') (id, 'i') (op, '=') (lit, '0') (delim, ';') ...
- ▶ Lexer provides a stream of tokens into the next stage: the parser.

Front end: Parser

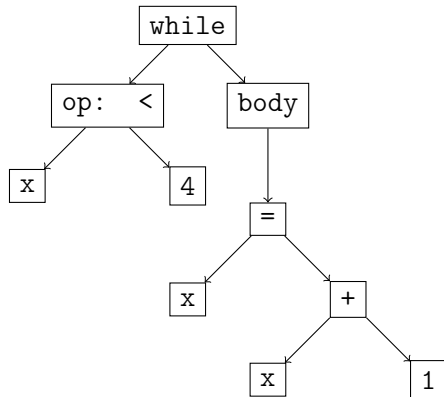
- ▶ The parser used to interpret the tokens and check they have the correct syntax.
- ▶ Builds an *abstract syntax tree*.
- ▶ The tree represents the structure of the program.
- ▶ Nodes for statements and expressions, and their arguments.
 - ▶ Statements are like a line in the program and don't return a value.
 - ▶ Expressions evaluate to a value.
- ▶ Structure of the program is encoded in the tree.

Front end: Abstract Syntax Tree: Expressions



Front end: Abstract Syntax Tree: Statements

```
while (x < 4) { x = x + 1; }
```



Front end: Recursive descent parser (1)

- ▶ Program language syntax defined by a grammar.
- ▶ Programs could begin with a statement.
- ▶ A statement could be an if, a while, ...
- ▶ E.g. An if statement has a condition, a true body, an else and a false body.
- ▶ Recursive descent parser would expand that statement into it's parts.
- ▶ Parser recursively calls itself, based on those structures.
- ▶ Abstract syntax tree built along the way.

Front end: Recursive descent parser (2)

```
ast_node statement() {  
    token t = gettok();  
    if (t == if_token) {  
        cond = condition();  
        true_body = statement();  
        expect(else_token);  
        false_body = statement();  
        return make_ast_if(cond, true_body, false_body);  
    }  
    else if (t == while_token) {...}  
    ...  
}
```

Front end: Symbol table

- ▶ Parser also constructs a symbol table.
- ▶ Symbol table contains all the variable names and information about them:
 - ▶ Type.
 - ▶ Scope (local, global).
- ▶ Could use a hierarchical data structure, or a stack to keep track of scope.
- ▶ Ensures local variables used over global with the same name.

Back end

Compiler back end

- ▶ Translates the intermediate representation into machine code.
- ▶ Targets specific architectures.
- ▶ Needs to output efficient code:
 - ▶ May do architecture specific optimisation.
 - ▶ Choose best instructions to use.
 - ▶ Choose best ordering of instructions.
- ▶ Needs to determine location of variables in memory: registers, stack, memory address, ...
- ▶ I.e., perform register allocation.

Back end: architectural optimisations

- ▶ A big topic.
- ▶ Pipelined processors help run multiple instructions at once.
- ▶ Ordering of those instructions important for performance.
- ▶ With out-of-order processors, getting a good instruction mix is key (Advanced Architecture course).
- ▶ Some instructions better than others, but do the same job.
- ▶ E.g. Memory loads of data:
 - ▶ Is the data aligned?
 - ▶ Does it span multiple cache lines?

Back end: code generation

- ▶ Finally, the compiler outputs the machine code for the program.
- ▶ Provides the object files in binary format.
- ▶ Often uses the linker to combine the object files together.

Bootstrapping

The first compiler

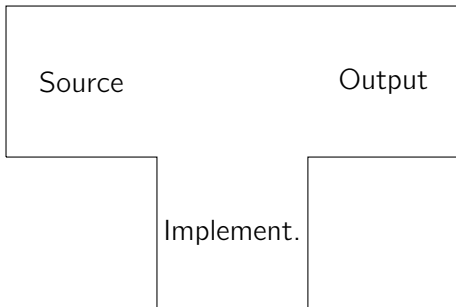
- ▶ How do you compile your compiler for a new language?
- ▶ How do you compile your compiler on a new architecture?

Bootstrapping

- ▶ *Bootstrapping* is the process for producing a compiler which can compile itself.
- ▶ I.e. How do you compile a C compiler which is written in C?
- ▶ The process results in an executable that can compile the source which generated that executable.

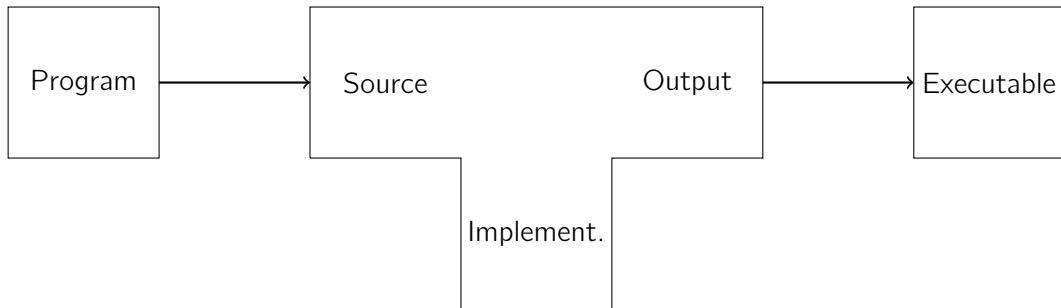
Tombstone diagrams

- ▶ Will use *Tombstone diagrams* (T-diagrams) to show the progress.
- ▶ Each T-diagram can be considered an executable.



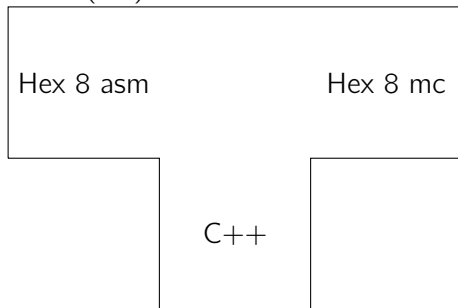
Tombstone diagrams

- ▶ Will use *Tombstone diagrams* (T-diagrams) to show the progress.
- ▶ Each T-diagram can be considered an executable.
- ▶ Represent the procedure for converting a program written in **Source** using a compiler written in an **Implementation** into the **Output**.



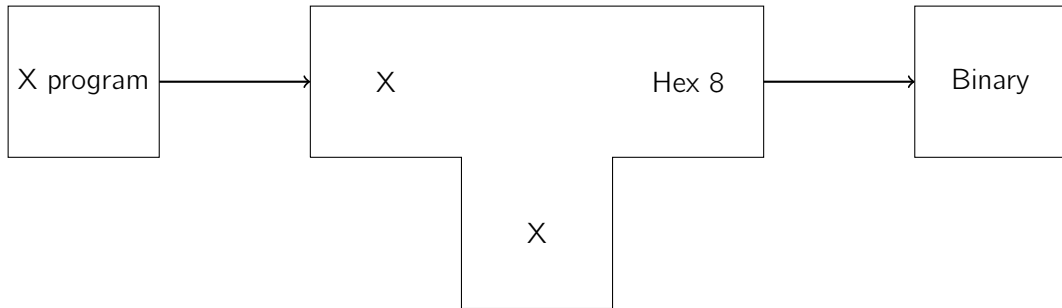
Tombstone diagrams: example

For example, my Hex 8 assembler is written in C++ and converts Hex 8 assembly (asm) to Hex 8 machine code (mc).

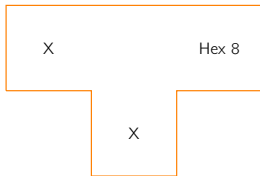


Bootstrapping compiler (1)

- ▶ Consider we need to bootstrap a X compiler, written in X, which produces machine code on an Hex 8 processor.
- ▶ This will give us an executable (shown as the T-diagram) which we can use to compile X programs to Hex 8 binaries.

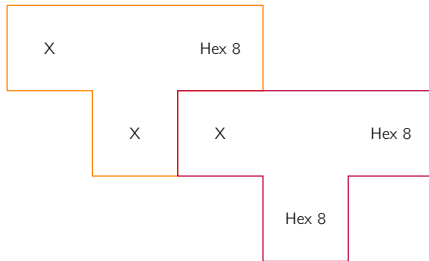


Bootstrapping compiler (2)



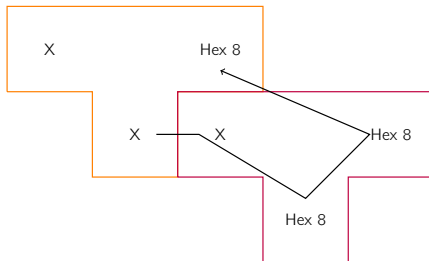
Bootstrapping compiler (2)

- ▶ Write another (minimal) **compiler** in Hex 8 machine code, which can compile X into Hex 8 machine code.



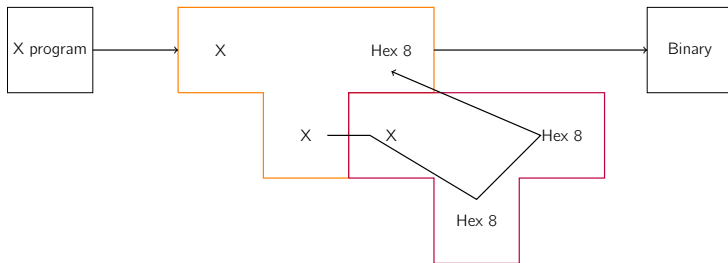
Bootstrapping compiler (2)

- ▶ Write another (minimal) **compiler** in Hex 8 machine code, which can compile X into Hex 8 machine code.
- ▶ Use this **compiler** to compile the first **compiler**.



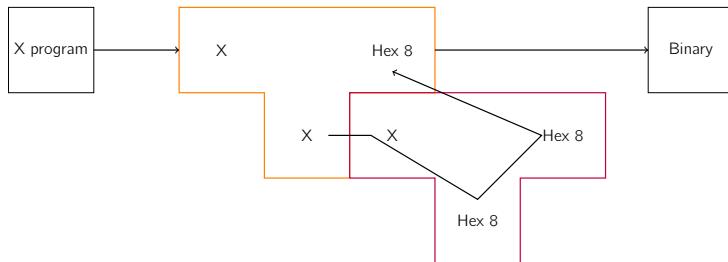
Bootstrapping compiler (2)

- ▶ Write another (minimal) **compiler** in Hex 8 machine code, which can compile X into Hex 8 machine code.
- ▶ Use this **compiler** to compile the first **compiler**.
- ▶ The result is a binary of the **compiler** which gives a path from X to Hex 8.



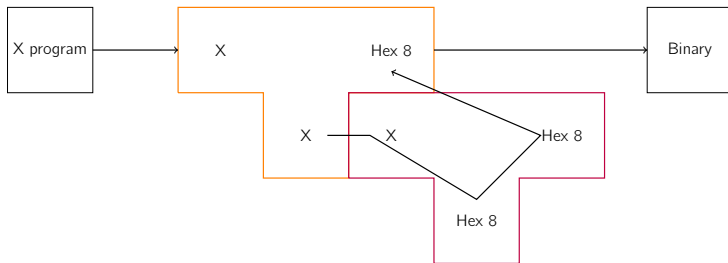
Bootstrapping compiler (2)

- ▶ Write another (minimal) **compiler** in Hex 8 machine code, which can compile X into Hex 8 machine code.
- ▶ Use this **compiler** to compile the first **compiler**.
- ▶ The result is a binary of the **compiler** which gives a path from X to Hex 8.
- ▶ Finally, use *that* binary to build the **compiler** written in X again.
- ▶ Results in a binary of the **compiler**.



Bootstrapping compiler (2)

- ▶ Write another (minimal) **compiler** in Hex 8 machine code, which can compile X into Hex 8 machine code.
- ▶ Use this **compiler** to compile the first **compiler**.
- ▶ The result is a binary of the **compiler** which gives a path from X to Hex 8.
- ▶ Finally, use *that* binary to build the **compiler** written in X again.
- ▶ Results in a binary of the **compiler**.
- ▶ No longer a need for the (minimal) **compiler** (X to Hex 8 written in Hex 8).



Bootstrapping compiler (3)

- ▶ The extra minimal compiler need only implement enough of the language to build the compiler.
- ▶ May need to be written by hand:
 - ▶ Using assembly.
 - ▶ Macro generator, using pattern substitution.
- ▶ That compiler is an executable, which can be used to compile the real compiler.
- ▶ The resulting binary can then build the compiler, given you a bootstrapped compiler.
- ▶ No need for that first minimal compiler any more.

Bootstrapping

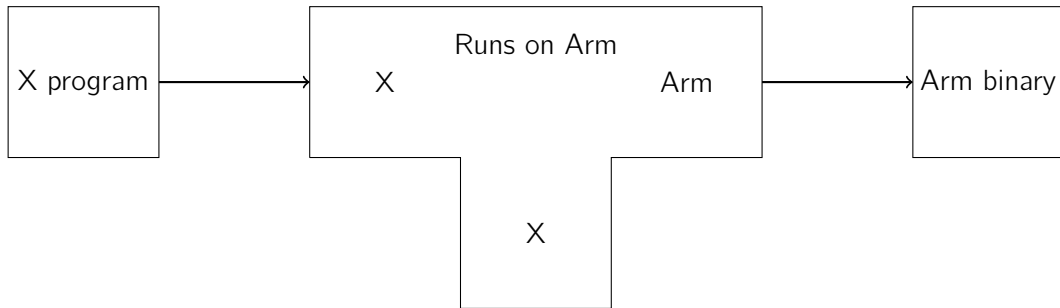
How do you build a compiler for a new language (in that language)?

- ▶ Write a tiny compiler in assembly/macros that builds a working subset of the language.
- ▶ Write a compiler in that subset, and use your compiler to compile it.

In short: write a compiler in one language, and use that to build the new compiler.

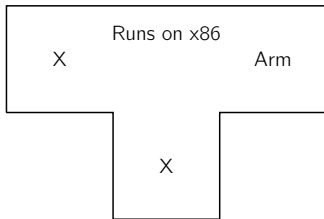
New architecture

- ▶ What about a compiler for a new architecture?
- ▶ That compiler executable needs to run natively on the new architecture.



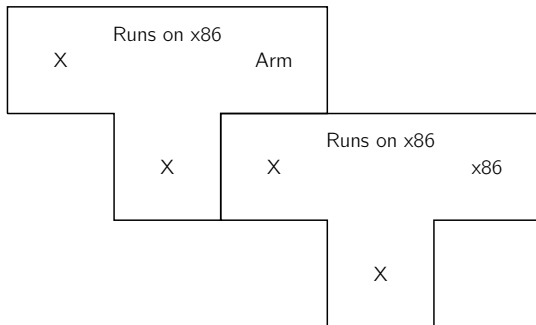
Cross-compiling (1)

- ▶ Take a similar approach to bootstrapping.
- ▶ Add a new back-end to our compiler to generate code for new architecture.



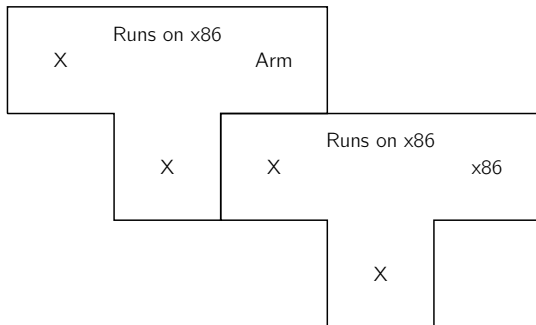
Cross-compiling (1)

- ▶ Take a similar approach to bootstrapping.
- ▶ Add a new back-end to our compiler to generate code for new architecture.
- ▶ Build it using the original compiler to get a binary which will:
 - ▶ Run on current architecture.
 - ▶ Build code for the new architecture.



Cross-compiling (1)

- ▶ Take a similar approach to bootstrapping.
- ▶ Add a new back-end to our compiler to generate code for new architecture.
- ▶ Build it using the original compiler to get a binary which will:
 - ▶ Run on current architecture.
 - ▶ Build code for the new architecture.
- ▶ A process known as *cross-compiling*.



Cross-compiling (2)

- ▶ Cross-compiler runs on old architecture and produces a binary which runs on the new architecture.
- ▶ Use the cross-compiler to build a compiler binary for the new architecture.
- ▶ Run that binary on the new architecture and compile the compiler natively.
- ▶ Results in a native compiler binary — no longer need the cross-compiler.

Cross-compiling

Don't confuse this with bootstrapping!

It's a similar idea but cross-compiling is building code that runs on one architecture on another.

E.g., use an x86 machine to build an Arm binary.

Further Reading

- ▶ *Compilers: Principles, Techniques, and Tools*: Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman.
- ▶ <https://llvm.org/docs/tutorial/MyFirstLanguageFrontend/index.html>
- ▶ <https://en.wikipedia.org/wiki/Compiler>
- ▶ https://en.wikipedia.org/wiki/Intermediate_representation
- ▶ https://en.wikipedia.org/wiki/Abstract_syntax_tree
- ▶ https://en.wikipedia.org/wiki/Recursive_descent_parser
- ▶ https://en.wikipedia.org/wiki/Symbol_table

Further Reading

- ▶ <https://llvm.org/pubs/2008-10-04-ACAT-LLVM-Intro.pdf>
- ▶ <http://www.aosabook.org/en/llvm.html>
- ▶ <http://web.cse.ohio-state.edu/software/2231/web-sw2/extras/slides/21.Abstract-Syntax-Trees.pdf>
- ▶ [https://en.wikipedia.org/wiki/Bootstrapping_\(compilers\)](https://en.wikipedia.org/wiki/Bootstrapping_(compilers))
- ▶ <http://people.cs.bris.ac.uk/~dave/boot.pdf>
- ▶ https://en.wikipedia.org/wiki/Tombstone_diagram